

Lab 02 – First Contact with OpenPLC – Solutions

Industrial Security (Bachelor)

• Maps to Section 02

• 30–45 min

LAB 02

🎯 Goal

Lecturer / TA reference for Lab 02. Lists the expected outputs of each step, explains **why** the holding registers stay zero, flags the recurring student mistakes, and gives a grading rubric.

✅ Step 1 – expected docker compose ps

After `docker compose up -d` from `bachelor/labs/_stack`, six services should report **Up** (the `openplc-init` sidecar is one-shot and exits 0):

Container	Status	Ports (host)
<code>is-landing</code>	Up	127.0.0.1:8000->80/tcp
<code>is-openplc</code>	Up (healthy)	127.0.0.1:8080->8080, 127.0.0.1:502->502
<code>is-openplc-init</code>	Exited (0)	—
<code>is-opcua</code>	Up	127.0.0.1:4840->4840
<code>is-mqtt</code>	Up	127.0.0.1:1883->1883
<code>is-student</code>	Up	—
<code>is-zeek</code>	Up	(shares <code>is-openplc</code> netns)

`is-openplc` reaches **healthy** within ~5–15s; `is-openplc-init` then runs, uploads `conveyor.st`, hits **Start PLC**, and exits within another ~10s.

✅ Step 3 – Monitoring page

On `http://127.0.0.1:8080/monitoring` the student should see:

- **Status:** Running (green).
- **Program:** `conveyor` (or `conveyor.st`).
- Variables `Start`, `Stop`, `Guard1`, `Guard2`, `Motor`, `Lamp` listed with live values. By default `Stop`, `Guard1`, `Guard2` read `TRUE`; `Start`, `Motor`, `Lamp` read `FALSE`.
- Forcing `Start := TRUE` latches `Motor` and `Lamp` to `TRUE` (seal-in); forcing `Stop := FALSE` drops them.
- **Cycle time:** typically **20–40 ms** on a Raspberry Pi class host (the IEC task interval is `T#20ms`; observed jitter on a busy laptop is well within this window). Anything `>100ms` suggests the host is overloaded or running on emulated `x86_64`.

✅ Step 4 – expected modbus_read.py output

```
1 $ docker compose exec student python3 /labs/scripts/modbus_read.py
2 Registers 0..9: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

Why all zero? The script reads ten *holding registers* (`%MW`, Modbus FC03) starting at address 0. `conveyor.st` only manipulates discrete I/O: it reads `Start/Stop/Guard*` from `%IX` (discrete inputs, FC02) and drives `Motor/Lamp` on `%QX` (coils, FC01/FC05). No `%MW` word is ever written, so every holding register stays at its power-on default of 0. This is the intended teaching moment: students must distinguish coils/discrete inputs from holding/input registers before Lab 03.

✓ Common student mistakes

- **Wrong working directory.** Running `docker compose up -d` from the repo root or from `bachelor/labs/02-ics-fundamentals/` fails with “*no configuration file provided*”. Compose only finds the stack from `bachelor/labs/_stack/`.
- **“I started the PLC manually and now upload fails”.** The `openplc-init` sidecar already did this. Re-uploading while the runtime is running returns an HTTP 500 on the OpenPLC web UI – stop the PLC first.
- **Connecting to port 502 from inside is-student.** Some students copy the host URL and try `127.0.0.1:502` from inside the student container; that loops back to the student container, not OpenPLC. The script correctly targets `172.28.0.10:502` (the OpenPLC service IP on the `is-lab` bridge).
- **Conversely from the host.** Running `nc 172.28.0.10 502` on the host fails – that subnet is internal to Docker. From the host use `127.0.0.1:502`; from any container use `172.28.0.10:502` or the DNS name `openplc`.
- **Port 502 already in use on the host.** On Linux, another Modbus tool (or a previous lab run with a stale container) may already hold `127.0.0.1:502`. `docker compose down` then `up -d` clears it.
- **Default OpenPLC credentials left unchanged.** Step 2 explicitly asks them to rotate the password. Check the deliverable screenshot – if it still says “*Welcome openplc*” with the stock UI hints, dock points.
- **Screenshot of an empty Monitoring page.** If they grab it before `Start PLC` ran, `Status` reads `Stopped` and no variables are listed. Ask for a re-take.
- **Cycle time of 0ms or blank.** Means the PLC is not actually running; usually the runtime crashed because they uploaded a syntactically broken `.st` from the optional Step 3 detour. Re-upload `conveyor.st`.

📋 Grading rubric

- **2 pts** – `docker compose ps` shows OpenPLC *Up (healthy)* and `openplc-init` exited 0.
 - **2 pts** – Default OpenPLC password changed; new password recorded in the lab notes.
 - **3 pts** – Monitoring screenshot shows **Status:** `Running`, program `conveyor`, and a plausible cycle time (20–40 ms).
 - **2 pts** – `modbus_read.py` output captured and matches `[0, 0, 0, 0, 0, 0, 0, 0, 0, 0]`.
 - **1 pt** – Short written explanation of *why* the registers are zero (holding registers vs. coils/discrete inputs). Award full credit only if both Modbus object types are named correctly.
- Total: **10 pts**. Deduct 1 pt if any deliverable is missing the observed cycle time; deduct 2 pts if the student exposed port 502 to a non-loopback interface (security regression).